Meet Google Play's target API level requirement

Release Notes

When you upload an APK, it must meet Google Play's target API level requirements (https://support.google.com/googleplay/android-developer/answer/11926878).

Starting August 31 2025:

- New apps and app updates must target Android 15 (API level 35) or higher to be submitted to Google Play; except for Wear OS, Android Automotive OS, and Android TV apps, which must target Android 14 (API level 34) or higher.
- Existing apps must target Android 14 (API level 34) or higher to remain available to new users on devices running Android OS higher than your app's target API level. Apps that target Android 13 (API level 33) or lower, including Android 12 (API level 31) or lower for Wear OS and Android TV, will only be available on devices running Android OS that are the same or lower than your app's target API level.

You will be able to request an extension to November 1, 2025 if you need more time to update your app. You'll be able to access your app's extension forms in Play Console later this year.

Exceptions to these requirements include:

- Permanently private apps that are restricted to users in a specific organization and intended for internal distribution only.

Note: Out-of-date apps are unavailable to new users of devices that run newer versions of Android. For more information, see the "App availability requirements" section in the Target API level requirements for Google Play apps (https://support.google.com/googleplay/android-developer/answer/11926878) article in the Play Console Help.

Why target newer SDKs?

Every new Android version introduces changes that bring security and performance improvements and enhance the Android user experience. Some of these changes only apply to apps that explicitly declare support through their `targetSdkVersion` manifest attribute (also known as the target API level).

Configuring your app to target a recent API level ensures that users can benefit from these improvements, while your app can still run on older Android versions. Targeting a recent API level also allows your app to take advantage of the platform's latest features (#modernize) to delight your users. Furthermore, as of Android 10 (API level 29), users see a warning (/about/versions/10/behavior-changes-all#low-target-sdk-warnings) when they start an app for the first time if the app targets Android 5.1 (API level 22) or lower.

This document highlights important points you need to know in updating your target API level to meet the Google Play requirement (https://support.google.com/googleplay/android-developer/answer/11926878). See the instructions in the following sections, depending on which version you are migrating to.

Note: If your Gradle file contains manifest entries, you can confirm or change the current value of **`targetSdkVersion`** in your app's Gradle file, as described in Configure your build (/studio/build#module-level). Alternatively, you can use the **`android:targetSdkVersion`** attribute in the manifest file, as described in the documentation for the **`<uses-sdk>`** (/guide/topics/manifest/uses-sdk-element) manifest element.

Migrate from Android 12 and higher (API level 31) to a more recent version

To update your app to target a more recent version of Android, follow the relevant behavior changes list:

- Android 13 behavior changes (/about/versions/13/behavior-changes-13)
- Android 14 behavior changes (/about/versions/14/behavior-changes-14)
- Android 15 behavior changes (/about/versions/15/behavior-changes-15)

- Android 16 behavior changes (/about/versions/16/behavior-changes-16)

Migrate from Android 11 (API level 30) to Android 12 (API level 31)

Security and Permissions

- Bluetooth (/guide/topics/connectivity/bluetooth/permissions): You must replace declarations for the `BLUETOOTH` (/reference/android/Manifest.permission#BLUETOOTH) and `BLUETOOTH_ADMIN` (/reference/android/Manifest.permission#BLUETOOTH_ADMIN) permissions with `BLUETOOTH_SCAN` (/reference/android/Manifest.permission#BLUETOOTH_SCAN), `BLUETOOTH_ADVERTISE` (/reference/android/Manifest.permission#BLUETOOTH_ADVERTISE), or `BLUETOOTH_CONNECT` (/reference/android/Manifest.permission#BLUETOOTH_CONNECT) permissions. You no longer need to make `LOCATION` runtime permission requests for Bluetooth operations.
- Location: Users can request apps to retrieve only approximate location information. You must request the `ACCESS_COARSE_LOCATION` (/reference/android/Manifest.permission#ACCESS_COARSE_LOCATION) permission any time you request `ACCESS_FINE_LOCATION` (/reference/android/Manifest.permission#ACCESS_FINE_LOCATION).
    - Intent filters: If your app contains activities (/guide/components/activities/intro-activities), services (/guide/components/services), or broadcast receivers (/guide/components/broadcasts) that use intent filters (/guide/components/intents-filters#Receiving), you must explicitly declare the android:exported (/guide/topics/manifest/activity-element#exported) attribute for these components.
- Hibernation: Apps may be put into hibernation mode if they are not used over a period of time. In hibernation mode your app's runtime permissions and cache are reset, and you can't run jobs or alerts. You can check your app's hibernation status (/topic/performance/app-hibernation).
- Pending intent mutability (/about/versions/12/behavior-changes-12#pending-intent-mutability): You must specify the mutability of each PendingIntent object that your app creates.

User Experience

- Custom notifications (/about/versions/12/behavior-changes-12#custom-notifications): Notifications with custom content views will no longer use the full notification area; instead, the system applies a standard template. This template ensures that custom notifications have the same decoration as other notifications in all states. This behavior is nearly identical to the behavior of `Notification.DecoratedCustomViewStyle`.
- Android App Links verification changes (/about/versions/12/behavior-changes-12#android-app-links-verification-changes): When using Android App Link verification, make sure that your intent filters include the BROWSABLE category and support the HTTPS scheme.

Performance

- Foreground service launch restrictions (/about/versions/12/behavior-changes-12#foreground-service-launch-restrictions): To target Android 12 or higher, your app can't start foreground services while it runs in the background, except for a few special cases. If an app attempts to start a foreground service while running in the background, an exception occurs (except for the few special cases).

  Consider using WorkManager to schedule and start expedited work (/topic/libraries/architecture/workmanager/how-to/define-work#expedited) while your app runs in the background. To complete time-sensitive actions that the user requests, start foreground services within an exact alarm.
- Notification trampoline restrictions (/about/versions/12/behavior-changes-12#notification-trampolines): When users tap notifications, some apps respond by launching an app component that starts the activity that the user sees and interacts with. This app component is known as a notification trampoline.

  Apps must not start activities from services or broadcast receivers that are used as notification trampolines. After a user taps on a notification or action button within the notification, your app cannot call `startActivity()` inside of a service or broadcast receiver.

View the complete set of changes that affect apps targeting Android 12 (API level 31) (/about/versions/12/behavior-changes-12).

Migrate from lower than Android 11 (API level 30)

Select the version of Android you will be migrating from:

Migrate to Android 5 (API level 21)

See the respective Behavior Changes page for each of the following releases to ensure your that your app has accounted for changes introduced in these releases:

- Android 5.0 (API level 21) (/about/versions/android-5.0)
- Android 4.4 (API level 19) (/about/versions/android-4.4).
- Android 4.1.x (API level 16) (/about/versions/android-4.1).

Continue by following the instructions in the next section.

Migrate to Android 6 (API level 23)

The following considerations apply to apps targeting Android 6.0 and higher versions of the platform:

- Runtime Permissions (/training/permissions/requesting)
    - Dangerous permissions are only granted at runtime. Your UI flows must provide affordances for granting these permissions.
    - Wherever possible, ensure your app is prepared to handle rejection of permission requests. For example, if a user declines a request to access the device's GPS, ensure your app has another way to proceed.

For an exhaustive list of changes introduced in Android 6.0 (API level 23), see the Behavior Changes (/about/versions/marshmallow/android-6.0-changes) page for that version of the platform.

Continue by following the instructions in the next section.

Migrate to Android 7 (API level 24)

The following considerations apply to apps targeting Android 7.0 and higher versions of the platform:

- Doze and App Standby

  Design for behaviors described in Optimizing for Doze and App Standby (/training/monitoring-device-state/doze-standby), which encompasses incremental changes introduced across several platform releases.

  When a device is in Doze and App Standby Mode, the system behaves as follows:

  - Restricts network access
  - Defers alarms, syncs, and jobs
  - Restricts GPS and Wi-Fi scans
  - Restricts normal-priority Firebase Cloud Messaging (https://firebase.google.com/docs/cloud-messaging) messages.

- Permission Changes

  - The system restricts access to app private directories.
  - Exposing a `file://` URI outside of your app triggers a `FileUriExposedException`. If you need to share files outside of your app, implement `FileProvider` (/training/secure-file-sharing/setup-sharing)

- The system forbids linking (/about/versions/nougat/android-7.0-changes#ndk) to non-NDK libraries.

For an exhaustive list of changes introduced in Android 7.0 (API level 24), see the Behavior Changes (/about/versions/nougat/android-7.0-changes) page for that version of the platform.

Continue by following the instructions in the next section.

Migrate to Android 8 (API level 26)

The following considerations apply to apps targeting Android 8.0 and higher versions of the platform:

- Background Execution Limits (/about/versions/oreo/background)

  - The system restricts services for apps not running in the foreground.

    - `startService()` (/reference/kotlin/android/content/Context#startService(android.content.Intent)) now throws an exception when an app tries to invoke it while `startService()` is prohibited.
    - To start foreground services, an app must use `startForeground()` (/reference/kotlin/android/app/Service#startForeground(int, android.app.Notification)) and `startForegroundService()` (/reference/kotlin/android/content/Context#startForegroundService(android.content.Intent)) .
    - Carefully review the changes made to the JobScheduler API, as documented on the Android 8.0 (API level 26) Behavior Changes page (/about/versions/oreo/android-8.0#jobscheduler).
    - Firebase Cloud Messaging (https://firebase.google.com/docs/cloud-messaging/) requires version 10.2.1 (https://developers.google.com/android/guides/releases#march_2017_-_version_1021) of the Google Play services SDK (https://developers.google.com/android/guides/overview), or higher.
    - When using Firebase Cloud Messaging (https://firebase.google.com/docs/cloud-messaging/), message delivery is subject to background execution limits. When background work is necessary upon message receipt, such as to perform background data sync, your app should schedule jobs using Firebase Job Dispatcher or JobIntentService instead. For more information, see the Firebase Cloud Messaging documentation (https://firebase.google.com/docs/cloud-messaging/).
- Implicit broadcasts
    - Implicit broadcasts are restricted. For information about handling background events, see the documentation for the `JobScheduler` (/reference/kotlin/android/app/job/JobScheduler) API.
- Background Location Limits
    - Apps running in the background have limited access to location data.
        - On devices with Google Play services, use the fused location provider (https://developers.google.com/android/reference/com/google/android/gms/location/FusedLocationProviderClient)

to get periodic location updates.

- Notification Channels (/about/versions/oreo/android-8.0#notifications)
  - You should define notification interruption properties (/training/notify-user/channels#importance) on a per-channel basis.
  - You must assign notifications to a channel for the notifications to appear.
  - This version of the platform supports `NotificationCompat.Builder` (/reference/kotlin/androidx/core/app/NotificationCompat.Builder).
- Privacy (/about/versions/oreo/android-8.0-changes#privacy-all)
  - ANDROID_ID (/reference/kotlin/android/provider/Settings.Secure#ANDROID_ID) is scoped per app signing key.

For an exhaustive list of changes introduced in Android 8.0 (API level 26), see the Behavior Changes (/about/versions/oreo/android-8.0-changes) page for that version of the platform.

Migrate from Android 8 (API 26) to Android 9 (API 28)

- Power Management (/about/versions/pie/power)
  - App Standby buckets (/about/versions/pie/power#buckets) bring new background restrictions based on app engagement, such as deferred jobs, alarms and quotas on high-priority messages
  - Battery saver improvements (/about/versions/pie/power#battery-saver) increase the limitations on app standby apps
- Foreground service permission (/about/versions/pie/android-9.0-changes-28#fg-svc)
  - Need to request the normal permission `FOREGROUND_SERVICE` (/reference/kotlin/android/Manifest.permission#FOREGROUND_SERVICE) (not runtime permission)
- Privacy changes (/about/versions/pie/android-9.0-changes-all#privacy-changes-all)
  - Limited access to background sensors (/about/versions/pie/android-9.0-changes-all#bg-sensor-access)
  - Restricted access to call logs, now in `CALL_LOG` (/reference/kotlin/android/Manifest.permission_group#CALL_LOG) permission group

    - Restricted access to phone numbers, requiring `READ_CALL_LOG` (/reference/kotlin/android/Manifest.permission#READ_CALL_LOG) permission
    - Restricted access to Wi-Fi information

For an exhaustive list of changes introduced in Android 9.0 (API level 28), see behavior changes (/about/versions/pie/android-9.0-changes-28).

Migrate from Android 9 (API level 28) to Android 10 (API level 29)

- Notifications with a full-screen intent (/training/notify-user/build-notification#urgent-message)
    - Need to request the normal permission `USE_FULL_SCREEN_INTENT` (/reference/android/Manifest.permission#USE_FULL_SCREEN_INTENT) (not runtime permission).
- Support for foldables (/guide/topics/ui/foldables) and large screen devices
    - Multiple activities can now be in the "resumed" state at the same time, but only one actually has focus.
        - This change affects `onResume()` (/reference/android/app/Activity#onResume()) and `onPause()` (/reference/android/app/Activity#onPause()) behavior.
        - New lifecycle concept of "topmost resumed" which can be detected by subscribing to `onTopResumedActivityChanged()` (/reference/android/app/Activity#onTopResumedActivityChanged(boolean)).
            - Only one activity can be "topmost resumed."
    - When `resizeableActivity` (/guide/topics/ui/multi-window#resizeableActivity) is set to `false`, apps can additionally specify a `minAspectRatio` (/reference/android/R.attr#minAspectRatio) which automatically letterboxes the app on narrower aspect ratios.
- Privacy changes (/about/versions/10/privacy/changes)
    - Scoped storage (/training/data-storage#scoped-storage)
        - External storage access is limited only to an app-specific directory and to specific types of media that the app has created.

- Restricted access to location while the app is in the background, requiring `ACCESS_BACKGROUND_LOCATION` (/reference/android/Manifest.permission#ACCESS_BACKGROUND_LOCATION) permission.
- Restricted access to non-resettable identifiers such as IMEI and serial number.
- Restricted access to physical activity information such as the user's step count, requiring `ACTIVITY_RECOGNITION` (/reference/android/Manifest.permission#ACTIVITY_RECOGNITION) permission.
- Restricted access to some telephony, Bluetooth, and Wi-Fi APIs (/about/versions/10/privacy/changes#location-telephony-bluetooth-wifi), requiring `ACCESS_FINE_LOCATION` (/reference/android/Manifest.permission#ACCESS_FINE_LOCATION) permission.
- Restricted access to Wi-Fi settings
  - Apps can no longer directly enable or disable Wi-Fi and need to do it using settings panels (/about/versions/10/features#settings-panels).
  - Restrictions on initiating a connection to a Wi-Fi network, requiring the use of either `WifiNetworkSpecifier` (/reference/android/net/wifi/WifiNetworkSpecifier) or `WifiNetworkSuggestion` (/reference/android/net/wifi/WifiNetworkSuggestion).

Migrate from Android 10 (API level 29) to Android 11 (API level 30)

- Privacy
  - Scoped storage enforcement (/about/versions/11/privacy/storage) : Apps should adopt the scoped storage model where app-specific, media, and other file types are saved and accessed using dedicated locations.
  - Permissions auto-reset (/about/versions/11/privacy/permissions#auto-reset): If users haven't interacted with an app for a few months, the system auto-resets the app's sensitive permissions. This shouldn't affect most apps. If your app primarily works in the background without user interactions, you may consider requesting users to disable auto reset. (/guide/topics/permissions/overview#auto-reset-permissions-unused-apps)
  - Background location access (/about/versions/11/privacy/location#background-location): Apps must request foreground and background location permission separately.

Granting access to background location permission can only be done in app settings (/training/location/permissions#request-background-location) instead of runtime permission dialogs.

- Package Visibility (/training/basics/intents/package-visibility-use-cases): When an app queries for the list of installed apps and services on the device, the returned list is filtered.
    - If you use Text-to-speech (/about/versions/11/behavior-changes-11#tts-engines) or Speech Recognition (/about/versions/11/behavior-changes-11#speech-recognition) services, you will need to add queries elements for services to the manifest file.

- Security
    - Compressed `resource.arsc` files are no longer supported (/about/versions/11/behavior-changes-11#compressed-resource-file)
    - APK Signature Scheme v2 now required. (/about/versions/11/behavior-changes-11#minimum-signature-scheme) For backward compatibility reasons, developers should also continue to sign with APK Signature Scheme v1.
    - Non-SDK interface restriction. Using non-SDK interfaces is not recommended for apps targeting API level 30, as some of these non-SDK interfaces are now blocked. See Non-SDK interfaces that are now blocked in Android 11 (/about/versions/11/non-sdk-11#new-blocked) for a comprehensive list of blocked non-SDK interfaces.

For an exhaustive list of changes introduced in Android 11 (API level 30), see the Behavior Changes (/about/versions/11/behavior-changes-11) page.

Continue to update to API 31 by following the instructions in the previous section (#pre12).

Modernize your apps

As you update the target API level for your apps, consider adopting recent platform features to modernize your apps and delight your users.

- Consider using CameraX (/camerax), which is in Beta, to make the most of using the camera.
- Use Jetpack (/jetpack) components to help you follow best practices, free you from writing boilerplate code, and simplify complex tasks so that you can focus on the code you care about.
- Use Kotlin (/kotlin) to write better apps faster, and with less code.
- Ensure you are following privacy (/privacy) requirements and best practices.
- Add dark theme (/guide/topics/ui/look-and-feel/darktheme) support to your apps.
- Add gesture navigation (/guide/navigation/gesturenav) support to your apps.
- Migrate your app (https://developers.google.com/cloud-messaging/android/android-migrate-fcm) from Google Cloud Messaging (GCM) to the latest version of Firebase Cloud Messaging.
- Take advantage of advanced window management.
    - Support larger aspect ratios (more than 16:9) to take advantage of recent advances in hardware. Ensure that your app resizes to fill the available screen space. Only declare a maximum aspect ratio as a last resort. For more information about maximum aspect ratios, see Declare Restricted Screen Support (/guide/practices/screens-distribution#MaxAspectRatio).
    - Add multi-window support (/guide/topics/ui/multi-window) to help your app increase productivity, and to manage multiple displays (/about/versions/oreo/android-8.0#mds).
    - If a great minimized app experience would improve the user experience, add support for Picture-in-Picture (/guide/topics/ui/picture-in-picture).
        - Optimize for devices with display cutout.
        - Don't assume status bar height. Instead, use `WindowInsets` (/reference/kotlin/android/view/WindowInsets) and `View.OnApplyWindowInsetsListener` (/reference/kotlin/android/view/View.OnApplyWindowInsetsListener). To learn more, see the droidcon NYC 2017 (https://www.youtube.com/watch?v=_mGDMVRO3iE) video. for an explanation.
        - Don't assume that the app has the entire window. Instead, confirm its location by using `View.getLocationInWindow()`.

(/reference/kotlin/android/view/View#getLocationInWindow(int%5B%5D)), not `View.getLocationOnScreen()` (/reference/kotlin/android/view/View#getLocationOnScreen(int%5B%5D)). * When handling `MotionEvent`, use `MotionEvent.getX()` (/reference/kotlin/android/view/MotionEvent#getX()) and `MotionEvent.getY()` (/reference/kotlin/android/view/MotionEvent#getY()), not `MotionEvent.getRawX()` (/reference/kotlin/android/view/MotionEvent#getRawX()), `MotionEvent.getRawY()` (/reference/kotlin/android/view/MotionEvent#getRawY()).

Check and update your SDKs and libraries

Make sure that your third-party SDK dependencies support API 31: Some SDK providers publish it in their manifest; others will require additional investigation. If you use an SDK that doesn't support API 31, make it a priority to work with the SDK provider to resolve the issue.

Additionally, note that your app or game's `targetSdkVersion` may restrict access to private Android platform libraries; see NDK Apps Linking to Platform Libraries (/about/versions/nougat/android-7.0-changes#ndk) for details.

You should also verify any restrictions that may exist in the version of the Android Support Library that you're using. As always, you must ensure compatibility between the major version of Android Support Library and your app's `compileSdkVersion`.

We recommend that you choose a `targetSdkVersion` smaller than or equal to the Support Library's major version. We encourage you to update to a recent compatible Support Library in order to take advantage of the latest compatibility features and bug fixes.

Test your app

After you update your app's API level and features as appropriate, you should test some core use cases. The following suggestions are not exhaustive, but aim to guide your testing process. We suggest testing:

- That your app compiles to API 29 without errors or warnings.
- That your app has a strategy for cases where the user rejects permission requests, and prompts the user for permissions. To do so:

    - Go to your app's App Info screen, and disable each permission.
    - Open the app and ensure no crashes.
        - Perform core use case tests and ensure required permissions are re-prompted.
- Handles Doze with expected results and no errors.
    - Using adb, place your test device into Doze while your app is running.
        - Test any use cases that trigger Firebase Cloud Messaging messages.
        - Test any use cases that use Alarms or Jobs.
        - Eliminate any dependencies on background services.
    - Set your app into App Standby
        - Test any use cases that trigger Firebase Cloud Messaging messages.
        - Test any use cases that use Alarms.
- Handles new photos / video being taken
    - Check that your app handles the restricted (/topic/performance/background-optimization#media-broadcasts) `ACTION_NEW_PICTURE` (/topic/performance/background-optimization#media-broadcasts) and (/topic/performance/background-optimization#media-broadcasts) `ACTION_NEW_VIDEO` (/topic/performance/background-optimization#media-broadcasts) broadcasts correctly (that is, moved to JobScheduler jobs).
    - Ensure that any critical use cases that depend on these events still work.
- Handles sharing files to other apps - Test any use case that shares file data with any other app (even another app by the same developer)
    - Test the content is visible in the other app and doesn't trigger crashes.

Further information

Opt in to emails in the Google Play Console (http://g.co/play/monthlynews) so that we can send you important updates and announcements from Android and Google Play, including our monthly partner newsletter.

Content and code samples on this page are subject to the licenses described in the Content License (/license). Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates.

Last updated 2025-07-31 UTC.